

Python pentru Începatori

Robert Zofei

2026-05-03

Table of contents

Prefață	3
Capitole	3
1 Structuri de decizie	4
1.1 Tipul de date Boolean	4
1.2 Operatorii de Comparatie (==, !=, <, >)	5
1.2.1 Diferenta dintre operatorul = si operatorul ==	6
1.3 Operatori Booleeni (and, or, not)	6
1.4 If/ Else	7
1.4.1 Instructiunea if	7
1.4.2 Instructiunea else	8
1.4.3 Instructiunea elif	9
2 Structuri repetitive	11
2.1 Bucle while	11
2.2 Instructiunea break	13
2.3 Instructiunea continue	13
2.4 Bucle for	14
2.5 Functia range()	14
2.6 Switch case	14
3 Functii	16
3.1 Instructiunea def	16
3.2 Instructiunea return	16
3.3 Valoarea none	16
3.4 Variabile locale si variabile globale	16
References	17

Prefață

Capitole

1. [Structuri de decizie](#)
2. [Structuri repetitive](#)
3. [Functii](#)

Această carte a fost realizată folosind Quarto.

Mai multe despre Quarto aici: <https://quarto.org/docs/books>.

1 Structuri de decizie

Astazi vom invata calculatorul sa ia decizii singur. Pentru a putea face asta, avem nevoie sa intelegem trei concepte: tipul de date Boolean, operatorii de comparatie, si instructiunile if/else.

1.1 Tipul de date Boolean

In tipurile de date cu care am lucrat pana acum, adica cele pentru numere si siruri de caractere, puteam sa memoram aproape orice numar sau text pe care ni le doream. Ei bine, variabilele de tip Boolean pot stoca doar doua valori: True sau False, adica adevarat sau fals. Doua lucruri importante: valorile acestea nu apar scrise intre ghilimele (cum erau string-urile), si incep mereu cu litera mare.

```
a = "text"
b = True
print(type(a))
print(type(b))
```

```
<class 'str'>
<class 'bool'>
```

Asta inseamna ca variabila **a** este de tip string, iar variabila **b** este de tip Boolean.

```
a = True
b = False
print(a,b)
```

```
True False
```

Poate ca va intrebati de unde vine aceasta denumire de Boolean. Acest tip de date este numit dupa matematicianul britanic George Boole. El a inventat algebra logica si a pus bazele functionarii tuturor calculatoarelor si a limbajelor de programare pe care le folosim noi astazi.

1.2 Operatorii de Comparatie (==, !=, <, >)

De multe ori, o decizie implica comparatia dintre doua sau mai multe valori. Pentru asta, avem niste operatori speciali:

Operator	Ce înseamnă
==	Egal cu
!=	Nu e egal cu
<	Mai mic decât
>	Mai mare decât
<=	Mai mic sau egal cu
>=	Mai mare sau egal cu

Haide sa incepem prin a compara niste numere intre ele (rezultatul va aparea sub bucata de cod):

```
1 == 1
```

True

```
4 == 32
```

False

```
2 != 3
```

True

```
2 != 2
```

False

```
64 == 64.0
```

True

Putem compara si bucati de text:

```
'salut' == 'salut'
```

True

```
'salut' == 'Salut'
```

False

Pana ca si valorile de adevar pot fi comparate intre ele:

```
True == True
```

True

```
True != False
```

True

1.2.1 Diferenta dintre operatorul = si operatorul ==

Acesti doi operatori sunt usor de confundat, iata ce trebuie sa retineti ca sa nu-i incurcati:

- Operatorul = pune valoarea din dreapta in variabila din stanga
- Operatorul == (egal cu) verifica daca doua valori sunt egale

Exemplu:

```
a = 2  
print(a == 2) # verifica daca valoarea din variabila 'a' este egala cu 2
```

True

1.3 Operatori Booleeni (and, or, not)

Avem, de asemenea, la dispozitia noastra si operatorii, **and**, **or**, si respectiv, **not**.

Operatorul **and** verifica daca absolut toate expresiile comparate sunt adevarate.

```
(1 < 5) and (2 < 5)
```

True

```
(1 < 5) and (2 < 5) and (10 < 5)
```

False

Vedem ca e de ajuns ca o singura expresie sa fie falsa ca rezultatul sa fie fals.

Operatorul **or** verifica daca cel putin una dintre expresii este adevarata.

```
(1 == 2) or (2 == 2)
```

True

Operatorul **not** neaga valoarea de adevar. Spre deosebire de **and** si **or**, **not** opereaza pe o singura expresie. Pur si simplu returneaza valoarea opusa.

```
not True
```

False

1.4 If/ Else

Vrem sa se execute o bucata de cod daca si nu mai daca se indeplineste o conditie, dar cum putem sa facem asta? Pai, cu ajutorul structurilor de decizie (**if**, **else**, **elif**):

1.4.1 Instructiunea if

Sa zicem ca avem un program care verifica daca numele cuiva este "Alin". Ne prefacem ca cineva a introdus mai devreme o valoare in variabila *nume*.

```
nume = "Alin"
if nume == "Alin":
    print("Salut, Alin!")
```

Salut, Alin!

În limbaj natural, am citi liniile de cod de mai sus astfel:

- În variabila **nume** se memorează textul “Alin”
- Dacă valoarea din variabila **nume** este aceeași sau este egală cu textul “Alin”, atunci se afișează pe ecran mesajul “Salut, Alin!”

Dacă expresia urmată de **if** este adevărată, sau, cu alte cuvinte, condiția a fost îndeplinită, atunci toate liniile de cod ce scrise cu alineat imediat sub instrucțiunea **if** se vor executa. Acest alineat, în lumea programării, se numește indentare, și se obține apăsând tasta **tab**. Dacă expresia se evaluează ca fiind falsă, însă, atunci calculatorul ignoră liniile de cod indentate aflate imediat sub **if**.

Instrucțiunea **if** conține:

1. Cuvântul cheie **if**
2. O condiție (adică o expresie care se evaluează ca fiind **True** sau **False**)
3. Două puncte :
4. Începând cu linia următoare, un bloc indentat de cod (numit și clauză **if**)

1.4.2 Instrucțiunea **else**

Să zicem că avem un site și vrem să verificăm dacă un utilizator și-a introdus parola corectă.

```
parola = "xyz"

if parola == "xyz":
    print("Acces permis.")
else:
    print("Ati introdus parola gresita.")
```

Acces permis.

În limbaj natural, am citi aceste linii de cod astfel:

- Parola introdusă de utilizator este **xyz**.
- Dacă parola este **xyz**, atunci se afișează pe ecran mesajul “Acces permis”.
- Altfel, se afișează mesajul “Ati introdus parola gresita.”

Instrucțiunea **else** conține:

1. Cuvântul cheie **else**
2. Două puncte :
3. Începând cu linia următoare, un bloc indentat de cod (numit și clauză **else**)

1.4.3 Instructiunea elif

Ar putea exista situatia in care trebuie sa verificam mai multe cazuri. In aceasta situatie folosim instructiunea **elif**, care inseamna **else if**. Aceasta instructiune poate fi folosita doar daca urmeaza dupa o instructiune **if** sau o alta instructiune **elif**.

Sa luam un exemplu in care verificam varsta cuiva:

```
varsta = 1520 # de ani

if varsta < 18:
    print("Esti copil.")
elif varsta > 1000:
    print("  Esti vampir.")
elif varsta > 18:
    print("Esti adult.")
```

Esti vampir.

Observati ca ordinea instructiunilor **elif** din acest exemplu este una avantajoasa. Daca inversam cele doua **elif**-uri, programul ar fi putut oferi raspunsuri gresite cand valoarea varstei depasea 1000. Ar fi spus ca o persoana de 1001 de ani este adult. Asta se intampla pentru ca expresiile se evalueaza pe rand de sus in jos. Odata ce una este corecta, se executa codul din clauza instructiunii respective, iar restul nu se mai verifica. De asemenea, bineinteles ca nu exista persoane de 1000 de ani sau vampiri (mai stii?).

```
# Varianta gresita
varsta = 1001 # de ani

if varsta < 18:
    print("Esti copil.")
elif varsta > 18:
    print("Esti adult.")
elif varsta > 1000:
    print("  Esti vampir.")
```

Esti adult.

Instructiunea **elif** contine:

1. Cuvantul cheie **elif**

2. O conditie (care se evalueaza cu True sau False)
3. Doua puncte :
4. Incepand cu linia urmatoare, un bloc indentat de cod (numit si clauza elif)

2 Structuri repetitive

Pana acum am invatat cum sa scriem programe formate din linii de cod care se executa, de sus in jos, o singura data. Dar in majoritatea programelor utile avem nevoie sa repetam instructiuni. Acest fapt ne permite sa automatizam lucrurile repetitive sau plictisitoare pe care le avem de facut. De exemplu: sa verificam fiecare obiect dintr-o lista, sa-i cerem unui utilizator sa introduca niste date pana cand o face in mod corect, sau sa facem un joc sa mearga pana cand jucatorul pierde.

In acest capitol vom invata urmatoarele concepte:

- Bucla **while** – repeta un bloc de cod cat timp conditia este adevarata (True).
- Instructiunea **break** – opreste executia unei bucle inainte de indeplinirea conditiei.
- Instructiunea **continue** – trece peste restul iteratiei curente si sare la urmatoarea.
- Bucla **for** – repeta un bloc de cod de un numar specific de ori, sau pentru fiecare obiect dintr-o lista.
- Functia **range()** – genereaza o secventa de numere. De obicei se foloseste pentru a se specifica de cate ori se repeta o bucla **for**.

2.1 Bucle while

Buclele **while**, (in engleza while loops) permit executarea in mod repetat a unei secvente de linii de cod cat timp se respecta o anumita conditie.

Ne uitam la aceste doua programe si nu dureaza mult sa observam ca arata aproape identic: singura diferenta dintre ele este ca in primul avem instructiunea **if**, iar in celalalt avem instructiunea **while**.

```
numar = 0
if numar < 3:
    print(numar)
    numar += 1
```

0

```
numar = 0
while numar < 3:
    print(numar)
    numar += 1
```

0
1
2

Avem o variabila numita **numar** in care stocam valoarea 0. Apoi urmeaza conditia: daca, **numar** este strict mai mic decat 3, atunci se afiseaza pe ecran numarul si ii crestem valoarea acestuia cu 1, adica din 0 devine $0 + 1 = 1$. In continuare, insa, apare diferenta:

- Daca conditia este respectata, clauza **if** (adica blocul de cod indentat aflat imediat sub instructiunea **if**) se executa o singura data .
- Codul din clauza **while** va continua sa se execute cat timp conditia este adevarata (True), mai exact cat timp **numar** este strict mai mic decat 3.

Instructiunea **while** este compusa din:

1. Cuvantul cheie **while**
2. O conditie (adica o expresie care se evalueaza ca True sau False)
3. Doua puncte :
4. Incepand cu linia urmatoare, un bloc indentat de cod (numit si clauza **while**)

Iata un program enervant, dar care ne ajuta sa intelegem mai bine cum functioneaza buclele **while**.

```
name = ""
while name != "cum te cheama":
    print("Te rog scrie cum te cheama")
    name = input()
print("Mersi!")
```

Te rog scrie cum te cheama
Rob
Te rog scrie cum te cheama
Robert
Te rog scrie cum te cheama
???????
Te rog scrie cum te cheama
cum te cheama
Mersi!

2.2 Instructiunea break

```
for number in range(10):  
    if number == 4:  
        break  
    print(number)
```

0
1
2
3

2.3 Instructiunea continue

```
for number in range(10):  
    if number % 2 == 0:  
        continue  
    print(number)
```

1
3
5
7
9

2.4 Bucle for

```
celebritati = ["Michael Jackson", "Selly", "Maria"]
for celebritate in celebritati:
    print(celebritate)
```

```
Michael Jackson
Selly
Maria
```

Afiseaza pe ecran fiecare celebritate din lista, cate una la fiecare iteratie.

2.5 Functia range()

```
for i in range (5):
    print(i)
```

```
0
1
2
3
4
```

2.6 Switch case

```
value = 2
match value:
    case 1:
        result = "one"
    case 2:
        result = "two"
    case 3:
        result = "three"
    case _:
        result = "unknown"
print(result)
```

two

3 Functii

Am folosit pana acum functiile `print()`, `len()`, `input()` si multe altele. Aceste functii sunt deja incluse in Python, dar stiati ca putem sa ne scriem propriile noastre functii?

Ce vom invata in acest capitol:

- instructiunea `def`
- instructiunea `return`
- valoarea `none`
- Variabile locale si variabile globale

3.1 Instructiunea `def`

3.2 Instructiunea `return`

3.3 Valoarea `none`

3.4 Variabile locale si variabile globale

References